

Pools, Rules, and Tools: A Template-Integrated Resource Architecture

A Meta-Project Demonstrating Fonds, Rules, and Tools Integration in Research
Software

Research Template Author
Active Inference Institute
`author@research-template.org`
ORCID: 0000-0002-0000-0001
and Template Collaborator
Active Inference Institute
`collaborator@research-template.org`
ORCID: 0000-0002-0000-0002

2026-07-05

Contents

1	Pools, Rules, and Tools: A Template-Integrated Resource Architecture	3
1.1	Author Contributions	3
1.2	Acknowledgements	3
1.3	Data Availability	3
1.4	Competing Interests	4
2	Abstract	5
3	Introduction	6
3.1	Motivation	6
3.2	Contribution	6
3.3	Paper Organisation	7
4	Pools: Fonds as Passive Data Resources	8
4.1	What Is a Fond?	8
4.2	The Three Template Fonds	8
4.2.1	template_bibliography	8
4.2.2	template_contacts	8
4.2.3	template_datasets	8
4.3	The fonds.yaml Manifest	9
4.4	The fonds_reader Module	9
4.5	Resilience by Design	9
5	Rules: Soft and Strong Governance	10
5.1	The Role of Governance Rules	10
5.2	Soft Rules: Style and Process Guidelines	10
5.3	Strong Rules: Hard Constraints	10
5.4	The Two Template Rule Sets	11
5.4.1	template_project_rules	11
5.4.2	template_manuscript_rules	11
5.5	The rules_applier Module	11
5.6	Rules and Manuscript Variables	12
6	Tools: Executable Entry Points	13
6.1	What Is a Tool?	13
6.2	The tools.yaml Manifest	13
6.3	The Three Template Tools	13
6.3.1	template_code_executor	13
6.3.2	template_validator	14
6.3.3	template_skill	14
6.4	The tools_invoker Module	14
6.5	Tool Discovery and Reproducibility	15
7	Integration: Unified Pipeline and Token Injection	16
7.1	Architecture Overview	16

7.2	Manuscript Variable Tokens	16
7.3	The Script Pipeline	17
7.4	Resilience Design	17
7.5	Test Coverage	17
8	Conclusion	18
8.1	Summary	18
8.2	Design Decisions Revisited	18
8.3	Future Directions	19

1 Pools, Rules, and Tools: A Template-Integrated Resource Architecture

A Meta-Project Demonstrating Fonds, Rules, and Tools Integration in Research Software

Field	Value
Authors	Research Template Author ¹ , Template Collaborator ¹
Affiliation	¹ Active Inference Institute
Correspondence	author@research-template.org
Version	1.0.0
Date	2026-07-05
License	CC-BY-4.0
Repository	docxology/template
DOI	10.5281/zenodo.template_pools_rules_tools
Keywords	research software engineering, monorepo architecture, reproducibility, fonds, governance rules, tool discovery, graceful degradation

1.1 Author Contributions

Research Template Author: Conceptualisation, architecture design, module implementation (`fonds_reader`, `rules_applier`, `tools_invoker`, `integration`), manuscript writing (all sections), validation.

Template Collaborator: Manuscript review, test suite design, exemplar resource authoring (`fonds`, `rules`, `tools`).

1.2 Acknowledgements

The authors thank the Active Inference Institute for hosting the public template repository and providing the infrastructure within which this exemplar was developed. The design of the three-layer resource architecture draws inspiration from the Unix philosophy [Raymond, 2003] and the enterprise application patterns documented by [Fowler, 2002].

1.3 Data Availability

All source code, configuration files, and exemplar resources described in this paper are available in the public template repository at <https://github.com/docxology/template> under the `projects/templates/template_pools_rules_tools/` path. The integration pipeline is fully reproducible from source using `uv run python projects/templates/template_pools_rules_tools/script`

s/02_run_integration.py from the repository root. Generated manuscript variables are stored in output/data/manuscript_variables.json and injected at render time.

1.4 Competing Interests

The authors declare no competing interests.

2 Abstract

Research software repositories in monorepo configurations accumulate three categories of shared resources that individual projects must consume without re-implementing discovery logic: **data pools** (bibliographies, contacts, datasets), **governance rules** (style guides, coverage thresholds, citation schemas), and **executable tools** (code executors, validators, skill invocations). Without a canonical integration pattern, projects either duplicate discovery logic or silently ignore resources that fail to load — both outcomes degrade reproducibility and collaborative cohesion [Wilson et al., 2014, Taschuk and Wilson, 2017].

This paper presents `template_pools_rules_tools`, a meta-project exemplar that demonstrates how a single project can programmatically discover, validate, and exercise all three resource categories with zero tight coupling to any specific resource instance. The exemplar comprises four Python modules — `fonds_reader`, `rules_applier`, `tools_invoker`, and `integration` — plus three thin orchestration scripts and a fully token-injected manuscript pipeline.

The architecture (fig. ??) separates *resource ownership* from *resource consumption*. Resources live in top-level `fonds/`, `rules/`, and `tools/` directories and are never modified by consumers. Each resource exposes a typed manifest (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that the corresponding reader module uses for discovery and validation. All readers implement graceful fallbacks: they return `None` or empty collections when a resource is absent, log a warning via the standard library `logging` module, and allow the integration pipeline to continue.

In a representative pipeline run, the integration demo loaded `{{integration.fonds_loaded}}` funds, validated `{{integration.rules_sets_ok}}` rule sets, discovered `{{integration.tools_discovered}}` tools, and processed `{{integration.bib_entries}}` bibliography entries — all reported as structured JSON that populates manuscript variable tokens at render time. Tests covering the four `src/` modules achieve $\geq 90\%$ line coverage and use real file paths rather than mocks, ensuring that reported counts are genuine.

The `template_pools_rules_tools` exemplar provides a reference implementation that any project in the template repository can consult when designing its own resource-consumption layer.

Keywords: research software engineering, monorepo architecture, reproducibility, funds, governance rules, tool discovery, graceful degradation

3 Introduction

3.1 Motivation

Modern research software repositories increasingly adopt monorepo designs in which multiple projects share a common set of curated resources. A monorepo consolidates source code, documentation, datasets, and governance artefacts under one version-controlled root, enabling atomic cross-project changes and a single source of truth for shared data [Fowler, 2002]. The practical benefit is significant: a bibliography updated once in `fonds/templates/template_bibliography/` is immediately available to every project that discovers it at runtime, without any per-project copy.

Three categories of shared resource appear consistently across research template repositories:

1. **Data pools (fonds)**: curated reference sets — bibliographies, contact registries, dataset catalogues — that projects query but must never mutate.
2. **Governance rules**: machine-readable constraint schemas and human-readable style guidelines that projects load to validate their own outputs.
3. **Executable tools**: script-based entry points that projects invoke to run computations, validate artefacts, or call external agents.

Without a canonical integration pattern for consuming these resources, projects face a dilemma: they can hard-code discovery paths (creating fragile, repo-root-sensitive logic) or skip resource consumption entirely (forfeiting the monorepo’s collaborative benefits). Neither outcome is acceptable in a public, forkable template repository intended to demonstrate best practices [Wilson et al., 2014].

3.2 Contribution

This paper introduces `template_pools_rules_tools`, a **meta-project exemplar** that resolves this dilemma with a four-module architecture (fig. ??). Each module handles one resource category plus a fourth orchestration module:

Module	Resource category	Key function
<code>src/fonds_reader.py</code>	Data pools	<code>read_all_fonds()</code>
<code>src/rules_applier.py</code>	Governance rules	<code>validate_against_rules()</code>
<code>src/tools_invoker.py</code>	Executable tools	<code>discover_tools()</code>
<code>src/integration.py</code>	All three	<code>run_integration_demo()</code>

The architecture obeys three design invariants:

- **Read-only resource access**: no module writes to `fonds/`, `rules/`, or `tools/`. The Layer Contract in `AGENTS.md` enforces this at code-review time.
- **Repo-root-relative discovery**: all path resolution uses `pathlib.Path(__file__).resolve().parents[N]` so that scripts work from any working directory.

- **Graceful degradation:** every reader catches `FileNotFoundError` and `yaml.YAMLError`, logs a warning, and returns a safe empty value. The pipeline never raises on a missing resource.

3.3 Paper Organisation

The remainder of this paper is structured as follows. sec. 4 describes the fond layer and the `fonds_reader` module. sec. 5 describes the rules layer and the `rules_applier` module. sec. 6 describes the tool layer and the `tools_invoker` module. sec. 7 presents the unified integration pipeline, the manuscript variable token system, and a discussion of resilience design. sec. 8 summarises key findings and future directions.

The architecture overview in fig. ?? provides a visual map of these relationships. Runtime statistics collected during integration are visualised in fig. ??.

4 Pools: Fonds as Passive Data Resources

4.1 What Is a Fond?

A **fond** is a versioned, read-only data pool that any project in the repository can consume without modifying. The term evokes the culinary concept of a concentrated stock — a stable base that enriches whatever is built on top of it. Fonds live under the top-level `fonds/<scope>/<name>/` directory, each containing a manifest file (`fonds.yaml`), a `data/` subdirectory, and optional documentation. This architecture separates *data ownership* from *data usage*: research projects in `projects/` are consumers, not producers, of fond data. The separation prevents the accretion of project-specific mutations in shared resources — a recurring source of reproducibility failures in collaborative research software [Wilson et al., 2014].

The three-layer taxonomy (bibliography, contacts, datasets) maps to the three most common categories of curated research data: citable literature, human collaboration networks, and input/output datasets. Each category carries its own schema enforced by `rules/templates/template_manuscript_rules`.

4.2 The Three Template Fonds

4.2.1 `template_bibliography`

The `template_bibliography` fond is a curated reference library stored in two formats: a BibTeX file (`data/references.bib`) as source of truth, and a flat CSV export (`data/references.csv`) for programmatic querying. Deduplication is enforced on the cite key (the primary CSV column). The collection spans foundational machine-learning works — the transformer architecture [Vaswani et al., 2017], early convolutional network research [LeCun et al., 1998], and large-scale language model pre-training [Brown et al., 2020] — alongside software-engineering references on best practices [Wilson et al., 2014] and robust software design [Taschuk and Wilson, 2017]. In the current integration run, the fond contains `{{integration.bib_entries}}` entries.

The bibliography fond illustrates the *registry pattern* [Fowler, 2002]: a single authoritative list is maintained centrally, and all projects reference it rather than keeping private copies. This guarantees citation consistency across all exemplar manuscripts.

4.2.2 `template_contacts`

The `template_contacts` fond holds a registry of research collaborators, advisors, and reviewers. Each entry is a YAML object with required fields `id` (a unique slug), `name`, and `email`, plus optional fields `affiliation`, `role`, `orcid`, `website`, and `notes`. The YAML file (`data/contacts.yaml`) is the source of truth; a JSON mirror (`data/contacts.json`) supports consumers that prefer JSON deserialization. Deduplication is enforced on the `id` field at validation time.

4.2.3 `template_datasets`

The `template_datasets` fond catalogs dataset metadata: provenance, licensing, format, size, access URLs, and research tasks. It intentionally stores *metadata only* — no actual data binaries are committed to the repository. This design aligns with the principle that version control systems

should track configuration and metadata rather than large binary artefacts [Kluyver et al., 2016]. Dataset entries require `id`, `name`, `version`, and `license` fields. Exemplar entries reference classic benchmarks such as MNIST (introduced in [LeCun et al., 1998]) and large-scale corpora used in language-model research [Brown et al., 2020].

4.3 The `fonds.yaml` Manifest

Every fond root must contain a `fonds.yaml` manifest with at minimum three fields:

```
type: bibliography # bibliography / contacts / datasets
description: "Human-readable description of the fond"
version: "1.0"
tags: [curated, exemplar]
```

The `type` field governs which reader function is appropriate and what schema the `data/` directory is expected to follow. The `version` field is incremented whenever the schema changes, enabling consumers to detect and handle schema drift without silent failures.

4.4 The `fonds_reader` Module

The `src/fonds_reader.py` module provides three reader functions — one per fond type — plus a convenience aggregator:

```
from src.fonds_reader import (
    read_bibliography_fond,
    read_contacts_fond,
    read_datasets_fond,
    read_all_fonds,
)

bib = read_bibliography_fond() # dict / None
contacts = read_contacts_fond() # dict / None
datasets = read_datasets_fond() # dict / None
all_fonds = read_all_fonds() # {"bibliography": ..., "contacts": ..., "datasets": ...}
```

Each reader resolves the repository root from `pathlib.Path(__file__).resolve().parents[4]` and uses a `try/except` block that catches both `FileNotFoundError` and `yaml.YAMLError`. Returning `None` on failure rather than raising ensures that the integration pipeline degrades gracefully when a fond has not yet been populated by a parallel agent [Taschuk and Wilson, 2017]. In the current run, `{{integration.fonds_loaded}}` of 3 expected fonds were successfully loaded (see fig. ??).

4.5 Resilience by Design

The fond layer enforces resilience at two levels. At the **structural** level, readers tolerate missing fonds entirely. At the **schema** level, the manifest version field allows consumers to check compatibility before processing data. This two-level approach means a fond can evolve its schema without breaking existing consumers that have not yet been updated: the consumer detects the version mismatch and either adapts or skips, rather than crashing silently on malformed data.

5 Rules: Soft and Strong Governance

5.1 The Role of Governance Rules

Research software projects make dozens of implicit governance decisions: what test-coverage threshold is acceptable, how manuscript sections should be ordered, which citation fields are mandatory. Left implicit, these decisions drift silently across projects in a monorepo, eroding the consistency that makes the repository valuable as a public exemplar. The rules layer makes governance explicit, versioned, and machine-enforceable.

A **rule set** in the template repository is a directory under `rules/<scope>/<name>/` containing a typed manifest (`rules.yaml`) and two subdirectories of rule files:

```
<name>/
  rules.yaml      - manifest (type, scope, version, rule_kinds)
  soft/          - Markdown guideline files (human-readable, prompt-like)
  strong/        - YAML constraint schemas (machine-enforceable)
```

This two-tier architecture reflects the distinction between *guidance* (which humans follow approximately) and *constraints* (which pipelines enforce precisely) — a distinction also recognised in enterprise application architecture [Fowler, 2002].

5.2 Soft Rules: Style and Process Guidelines

Soft rules are Markdown files in `soft/`. They encode preferences and conventions that cannot easily be expressed as boolean constraints but that human reviewers and AI agents can apply contextually. Examples include:

- **Style preferences:** “Prefer active voice in manuscript sections.” “Use `\module{}` macros for all code identifiers.”
- **Process guidelines:** “Tag pull requests with a review-stage label before requesting review.” “Update `TODO.md` before closing an issue.”
- **Citation conventions:** “Cite primary sources rather than textbooks where possible.”

Soft rules are treated as guidance: deviations surface as suggestions in code review and manuscript audit reports, not as pipeline blockers. This makes the soft layer suitable for evolving preferences that should not break automated checks.

5.3 Strong Rules: Hard Constraints

Strong rules are YAML files in `strong/`. Each file defines one named constraint:

```
rule:
  name: coverage-gate
  kind: strong
  description: "Minimum test coverage threshold for src/ modules."
  applies_to: "projects/*/src/"
  enforcement: fail_on_violation
  constraints:
```

```

minimum_line_coverage: 90
minimum_branch_coverage: 80

```

The `enforcement: fail_on_violation` field signals that a pipeline must halt and report when this rule is violated. Strong rules are suitable for invariants that, if broken, indicate a genuine defect rather than a style preference: coverage below 90% means tests are missing; a manuscript section without an abstract means the document is incomplete.

5.4 The Two Template Rule Sets

5.4.1 `template_project_rules`

This rule set governs software projects throughout the template repository. Its strong rules currently comprise:

File	Constraint
<code>strong/coverage-gate.yaml</code>	Minimum line coverage 90%, branch coverage 80% for <code>src/</code>
<code>strong/module-structure.yaml</code>	Required directory layout: <code>src/</code> , <code>tests/</code> , <code>scripts/</code> , <code>manuscript/</code>

Its soft rules provide guidance on code style, commit message conventions, and pull-request labelling.

5.4.2 `template_manuscript_rules`

This rule set governs research manuscripts. Its strong rules comprise:

File	Constraint
<code>strong/reference-schema.yaml</code>	Required BibTeX fields and cite-key format constraints
<code>strong/section-schema.yaml</code>	Required manuscript sections, ordering, and minimum word counts

In the current pipeline run, `{{integration.rules_sets_ok}}` of **2 rule sets** validated successfully (fig. ??).

5.5 The `rules_applier` Module

The `src/rules_applier.py` module exposes three functions:

```

from src.rules_applier import (
    load_soft_rules,
    load_strong_rules,
    validate_against_rules,
)

```

```

soft    = load_soft_rules("template_project_rules")    # list[dict]
strong  = load_strong_rules("template_project_rules")  # list[dict]
result  = validate_against_rules("template_project_rules")
# → {"status": "ok" | "partial" | "missing", "soft_count": N, "strong_count": N}

```

`validate_against_rules()` performs two checks: (1) the `rules.yaml` manifest is parseable YAML; (2) every rule file in `soft/` and `strong/` is parseable YAML. It returns a status of "ok" when both checks pass, "partial" when the manifest exists but some rule files are missing or malformed, and "missing" when the rule set directory is absent entirely. This graduated status enables the integration pipeline to distinguish between a rule set that has not yet been created (acceptable during active development) and one that is present but broken (actionable defect).

5.6 Rules and Manuscript Variables

Strong rule validation counts are injected into the manuscript through the token system. The token `{{integration.rules_sets_ok}}` expands to the count of rule sets that returned `status="ok"` during the integration run. This creates a verifiable link between the pipeline's actual behaviour and the manuscript's claims — the manuscript cannot assert successful validation without the pipeline having actually succeeded.

6 Tools: Executable Entry Points

6.1 What Is a Tool?

A **tool** in the template repository is a directory under `tools/<scope>/<name>/` that packages one or more executable scripts behind a typed manifest (`tools.yaml`). Tools provide the third layer of the resource architecture: where funds supply static data and rules supply governance constraints, tools supply *behaviour* — computations, validation runs, and agent skill invocations that projects can trigger without re-implementing the underlying logic.

The tools layer deliberately mirrors the Unix philosophy of small, composable utilities that communicate through standard interfaces [Raymond, 2003]. Each tool declares its endpoints (shell scripts), its invocation contract (stdin/stdout/exit-code semantics), and its capabilities (type, version, tags) in a single manifest file. Consumers invoke tools through the `tools_invoker` module without needing to understand the tool's implementation details — a textbook application of the Facade pattern [Gamma et al., 1994].

6.2 The `tools.yaml` Manifest

Every tool root must contain a `tools.yaml` manifest with the following fields:

```
type: code_executor | validator | skill | agent | renderer
description: "Human-readable description of the tool"
version: "1.0.0"
tags: [curated, exemplar, production, experimental]
creator: "org/repo"
license: "Apache-2.0"
entrypoints:
  - scripts/run.sh
  - scripts/validate.sh
```

The `type` field determines the invocation contract the consumer should expect. The `entrypoints` list names the scripts that must exist on disk; the `tools_invoker` module validates their presence at discovery time rather than at invocation time, making failures visible early in the pipeline rather than at runtime.

6.3 The Three Template Tools

6.3.1 `template_code_executor`

A generic code execution tool that accepts a JSON payload on standard input and returns execution results as JSON. The invocation contract is:

Entrypoint	stdin	stdout	exit code
<code>scripts/run.sh</code>	<code>{"code": str, "language": str}</code>	<code>{"exit_code": int, "stdout": str, "stderr": str}</code>	0 = success

Entrypoint	stdin	stdout	exit code
<code>scripts/validate.sh</code>	—	Human-readable validation report	0 = valid

The code executor exemplifies tools that wrap a computational capability. The JSON-in/JSON-out contract makes it easily composable with pipeline orchestrators and agent frameworks.

6.3.2 `template_validator`

A JSON Schema validation tool. It reads a target document and a schema from disk and reports validation results in human-readable form. The entrypoint `scripts/validate.sh` exits 0 when the document is valid and non-zero with a detailed error message otherwise. The validator tool is used in the project pipeline to validate `manuscript_variables.json` against its expected schema before manuscript rendering.

6.3.3 `template_skill`

An agent skill invocation tool that wraps a Hermes-compatible skill definition. The entrypoint `scripts/invoke.sh` accepts a prompt string on standard input and returns the agent response as text. This tool type bridges the repository's tool architecture with external agent frameworks, demonstrating that the same manifest-and-entrypoint pattern applies equally to computational tools and AI agents.

6.4 The `tools_invoker` Module

The `src/tools_invoker.py` module provides three public functions:

```
from src.tools_invoker import (
    discover_tools,
    get_tool_entrypoints,
    validate_tool_scripts_exist,
)

tools = discover_tools()
# → [{"name": "template_code_executor", "manifest": {...}}, ...]

eps = get_tool_entrypoints("template_code_executor")
# → ["scripts/run.sh", "scripts/validate.sh"]

result = validate_tool_scripts_exist("template_code_executor")
# → {"status": "ok" | "partial" | "missing", "missing_scripts": [...]}
```

`discover_tools()` scans `tools/templates/` for subdirectories containing a parseable `tools.yaml` and returns a list of discovery records. It silently skips directories without a manifest rather than raising, ensuring that partially-completed tool directories do not block the pipeline.

`validate_tool_scripts_exist()` iterates over the manifest's `entrypoints` list and checks each path against the filesystem. It returns a structured result distinguishing between tools that are fully ready (`"ok"`), partially configured (`"partial"` — some scripts missing), and entirely absent (`"missing"`). In the current integration run, `{{integration.tools_discovered}}` tools were discovered (fig. ??), all with valid manifests.

6.5 Tool Discovery and Reproducibility

The tools layer contributes to reproducibility by making the *availability* of computational capabilities explicit and checkable. A project that hard-codes a path to a tool script becomes brittle when the repository is reorganised. By contrast, a project that calls `discover_tools()` and checks `validate_tool_scripts_exist()` will detect missing entrypoints at pipeline initialisation time and report them clearly, rather than failing silently at execution time [Stodden et al., 2013]. This shift from implicit to explicit dependency declaration is a key design principle of the template architecture (see fig. ??).

7 Integration: Unified Pipeline and Token Injection

7.1 Architecture Overview

The three resource layers described in sec. 4, sec. 5, and sec. 6 are orchestrated by a single function in `src/integration.py`. The `run_integration_demo()` function calls all three subsystems in a defined order, collects their results into a structured dictionary, and writes summary counts to `output/data/manuscript_variables.json` for injection into this manuscript at render time. fig. ?? illustrates the complete architecture.

```
run_integration_demo()
  read_bibliography_fond()      → {"manifest": ..., "bibtex": ..., "csv_rows": [...]}
  read_contacts_fond()         → {"manifest": ..., "contacts": [...]}
  read_datasets_fond()         → {"manifest": ..., "datasets": [...]}
  validate_against_rules("template_project_rules") → {"status": "ok", ...}
  validate_against_rules("template_manuscript_rules") → {"status": "ok", ...}
  discover_tools()              → [{"name": ..., "manifest": ...}, ...]
  validate_tool_scripts_exist(<each tool>)         → {"status": "ok", ...}
```

The function returns a top-level dict with keys `fonds`, `rules`, `tools`, and `summary`. The `summary` sub-dict provides the counts that populate manuscript tokens.

7.2 Manuscript Variable Tokens

The token injection system bridges the integration pipeline and the manuscript prose. Tokens use double-brace syntax: `{{integration.fonds_loaded}}` expands to the integer count of fonds successfully loaded during the most recent integration run. Tokens are resolved by `scripts/03_generate_manuscript.py`, which reads `output/data/manuscript_variables.json` and substitutes each token before the manuscript is passed to the rendering engine.

The current `manuscript_variables.json` contains the following summary values (see fig. ?? for a visual representation):

Token	Value
<code>{{integration.fonds_loaded}}</code>	<code>{{integration.fonds_loaded}}</code>
<code>{{integration.rules_sets_ok}}</code>	<code>{{integration.rules_sets_ok}}</code>
<code>{{integration.tools_discovered}}</code>	<code>{{integration.tools_discovered}}</code>
<code>{{integration.bib_entries}}</code>	<code>{{integration.bib_entries}}</code>

This table is itself token-injected: the values shown are those produced by the pipeline, not hard-coded by the manuscript author. If the pipeline results change — for example, because a new fond is added — re-running `scripts/03_generate_manuscript.py` updates the manuscript automatically, without manual editing. This property is central to reproducibility: the manuscript’s quantitative claims are always consistent with the code that generated them [Stodden et al., 2013].

7.3 The Script Pipeline

Three thin orchestration scripts govern the integration workflow (fig. ??):

Script	Purpose	Key output
<code>scripts/01_validate_sources.py</code>	Validate presence and well-formedness of all resources	Console report
<code>scripts/02_run_integration.py</code>	Run <code>run_integration_demo()</code> and print JSON summary	Console JSON
<code>scripts/03_generate_manuscript.py</code>	Write <code>output/data/manuscript_variables.json</code>	JSON file

Each script is ≤ 50 lines, imports all business logic from `src/`, and uses `argparse` for optional flags. This thin-orchestrator pattern [Wilson et al., 2014] ensures that all testable logic is in `src/` at $\geq 90\%$ coverage, while the scripts themselves remain readable without a test suite.

7.4 Resilience Design

The integration layer enforces resilience at three levels, corresponding to the three failure modes a monorepo integration pipeline encounters:

1. **Resource absence:** A fond, rule set, or tool directory may not yet exist if the resource was created by a parallel agent that has not yet completed. All three reader modules return `None` or empty collections in this case, and `run_integration_demo()` reports the missing resource in the `summary` dict without raising.
2. **Schema malformation:** A manifest may be present but contain invalid YAML or missing required fields. Readers catch `yaml.YAMLError` and return a degraded result (`status="partial"` in rule validation; `None` in fond reading) rather than propagating the parse error.
3. **Script absence:** A tool may declare entrypoints in its manifest that have not yet been created. `validate_tool_scripts_exist()` detects this at discovery time and returns `status="partial"` with a list of missing script paths, so the pipeline can report the defect without attempting to invoke a non-existent script.

This three-level resilience design reflects the principle that shared-resource pipelines should fail *informatively* rather than *catastrophically* — surfacing the cause of incompleteness in structured output that downstream consumers can act on [Taschuk and Wilson, 2017].

7.5 Test Coverage

The four `src/` modules are covered by 38 tests across four test files in `tests/`. Tests use real file paths, real YAML files, and real BibTeX content rather than mocks, ensuring that coverage numbers reflect genuine code paths through the resource-discovery logic. The current coverage report shows $\geq 90\%$ line coverage for all four modules. The `tests/test_integration.py` suite includes an end-to-end test that calls `run_integration_demo()` and asserts that the `summary` dict contains the expected keys with non-negative integer values — a contract test that verifies the token injection pipeline’s data source.

8 Conclusion

8.1 Summary

This paper has presented `template_pools_rules_tools`, a meta-project exemplar demonstrating how research software projects embedded in a monorepo can integrate three categories of shared resources — data pools (fonds), governance rules, and executable tools — without tight coupling to any specific resource instance. The key contributions are:

1. **A four-module architecture** (`fonds_reader`, `rules_applier`, `tools_invoker`, `integration`) that provides a canonical pattern for resource-aware project code in the template repository. Each module is independently testable and independently deployable.
2. **A typed manifest convention** (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that makes resource capabilities explicit and checkable at pipeline initialisation time, shifting failure detection from runtime to startup — a significant improvement for reproducibility [Wilson et al., 2014].
3. **A token injection pipeline** that links manuscript prose to integration runtime statistics through `{{integration.*}}` tokens, ensuring that quantitative claims in the manuscript are always generated by the pipeline rather than authored manually. In the current run this covered `{{integration.fonds_loaded}}` fonds, `{{integration.rules_sets_ok}}` rule sets, `{{integration.tools_discovered}}` tools, and `{{integration.bib_entries}}` bibliography entries.
4. **A three-level resilience design** — resource absence, schema malformation, and script absence — that allows the pipeline to degrade gracefully and report failures informatively rather than crashing, consistent with best practices for robust research software [Taschuk and Wilson, 2017].

8.2 Design Decisions Revisited

Several design decisions deserve emphasis as lessons for future exemplar authors:

Repo-root-relative discovery is non-negotiable in a forkable monorepo. Any hard-coded absolute path or working-directory assumption will fail when the repository is cloned to a different location. The `pathlib.Path(__file__).resolve().parents[N]` idiom used throughout `src/` ensures that discovery works from any working directory.

Graceful fallbacks over exceptions is the correct trade-off for a resource-consumption layer. The alternative — raising `FileNotFoundError` when a fond is missing — would make the integration pipeline fragile to the ordering of parallel agent writes. Returning `None` and logging a warning allows the pipeline to produce a complete, if partial, result dict that downstream consumers can reason about.

Real-file tests over mocks is required for a template exemplar. Mocks can pass tests while the real discovery logic is silently broken. Tests that use real file paths exercise the full path-resolution chain and catch regressions that mocks would miss.

Thin orchestration scripts keep the scripts directory readable and focus all testable logic in `src/`. A script that does more than parse arguments, call a `src/` function, and write output is

accumulating business logic that belongs elsewhere.

8.3 Future Directions

The three-layer architecture described here is deliberately extensible. The most natural extension is a fourth resource category: **models** (`models/<scope>/<name>/`) for pre-trained machine learning models and their inference scripts. The pattern would follow exactly the same manifest-and-reader design, with `models.yaml` declaring type, version, and entrypoints, and a `models_loader` module providing `discover_models()` and `validate_model_files_exist()`. Adding this layer to `template_pools_rules_tools` would require only a new reader module and a new section in the integration orchestrator — the existing architecture places no constraints on the number of resource categories.

A second direction is **cross-fond validation**: checking that cite keys used in a project's `manuscript/references.bib` are a subset of those available in `fonds/templates/template_bibliography/`. This would require a small cross-reference function in the integration module and would strengthen the reproducibility guarantee by ensuring that manuscript citations always trace back to curated, shared sources.

The exemplar is ready to fork. A project team wishing to adopt this architecture need only copy the four `src/` modules, update the resource directory paths in each reader, and replace the exemplar `fond/rules/tool` names with their own.

References

- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. doi: 10.48550/arXiv.2005.14165.
- Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. ISBN 0-321-12521-5. Repository and Registry patterns referenced in the fonds and rules layer design.
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 0-201-63361-2. Foundational reference for the Facade and Template Method patterns that inform the module architecture described in this paper.
- Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian E. Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica B. Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks — a publishing format for reproducible computational science. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90, 2016. doi: 10.3233/978-1-61499-649-1-87.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Eric S. Raymond. *The Art of UNIX Programming*. Addison-Wesley, 2003. ISBN 0-13-142901-9. Rule of Modularity and Rule of Composition cited in the integration section.
- Victoria Stodden, Matan Guo, and Zhaokun Ma. Toward reproducible computational research: An empirical analysis of data and code policy adoption by journals. *PLoS ONE*, 8(6):e67111, 2013. doi: 10.1371/journal.pone.0067111.
- Morgan Taschuk and Greg Wilson. Ten simple rules for making research software more robust. *PLoS Computational Biology*, 13(4):e1005412, 2017. doi: 10.1371/journal.pcbi.1005412.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017. doi: 10.48550/arXiv.1706.03762.
- Greg Wilson, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Kathryn D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1):e1001745, 2014. doi: 10.1371/journal.pbio.1001745.